PROYECTO FINAL DE CICLO

DASHBOARD MONITORIZACIÓN DE UN SERVIDOR

Jorge Pizarro Gómez

CFP/UAX
Centro de Formación Profesional

Tutora: Numilén Joost Newbery

Índice

Abstract 4
Resumen 5
Justificación del proyecto 6
Introducción 8
Objetivos 9
R – Requisitos 9
F – Funciones 9
T – Tareas 10
P – Pruebas 11
Descripción 12
Uso del proyecto 12
Plan de desarrollo del proyecto 12
Fase 1: Scripts Bash 13
Fase 2: Base de datos en MySQL 23
Fase 3: Panel Web 29
Fase 4: Conexión a base de datos 31
Tecnologías 32
Servidor Ubuntu 32
Descripción 32
Uso en el proyecto 33
Bash y Shell Scripting 33
Descripción 33
Uso en el proyecto 34
Dbeaver 35
Descripción 35
Uso de la herramienta en el proyecto 36
Apache Server 36
Descripción 36
Uso en el proyecto 38
PHP 38
Descripción 38
Uso de la herramienta en el proyecto 40
DHTML 40
HTML5 40
Descripción 40

Uso de la herramienta en el proyecto 42
CSS 42
Descripción 42
Uso de la herramienta en el proyecto 43
JavaScript 43
Descripción 43
Uso de la herramienta en el proyecto 44
Bootstrap 44
Descripción 44
Uso de la herramienta en el proyecto 44
Tabla resumen de las tecnologías 45
Metodología 45
Trabajos futuros 49
Conclusiones 51
Bibliografía 53
Bibliografía anexo 2 55

Abstract

A web-based admin panel is proposed to easily and quickly visualize network system monitoring, server resources, and services. Designed for system administrators, the platform will store logs in a database for performance analysis and management.

The development consists of four phases:

Bash Scripts (Linux) – Scripts will be created to gather and monitor server resources and services, including Apache status and HTTP traffic.

Database (MySQL, DBeaver) – A database will be designed to store and manage collected data.

Web Dashboard (HTML, CSS, JavaScript, PHP) – An interactive dashboard will be developed, hosted on an Apache web server.

Integration – The database and dashboard will be connected to ensure real-time data visualization.

This project provides an efficient tool for server administration, improving performance monitoring and issue detection.

Resumen

Se propone crear un panel de administración web en el que se pueda visualizar fácil y rápidamente información relativa a la monitorización de sistemas de red, diversos recursos y servicios de un servidor. El portal web estará diseñado para que un administrador de sistemas tenga acceso con facilidad a la información del servidor administrado. Además, la plataforma estará diseñada para almacenar registros en una base de datos, facilitando el análisis y la gestión del rendimiento del sistema.

El desarrollo se estructura en cuatro fases. En primer lugar, se crearán disintos scripts, en un entorno Linux, cuya tarea será proporcionar la información relativa a los distintos recursos y servicios del servidor que se pretenden monitorear y analizar.

En la segunda fase, se diseñará una base de datos en MySQL, utilizando Dbeaver, para almacenar y gestionar la información recopilada y aportada por los scripts desarrollados durante la primera fase.

La tercera fase contempla la creación de un dashboard web interactivo utilizando tecnologías tales como: HTML y CSS para el diseño, JavaScript para la interactividad, y PHP para la integración de los scripts con la base de datos. Se empleará el servidor web Apache para alojar la plataforma y gestionar las solicitudes del usuario.

Finalmente, se establecerá la conexión entre la base de datos y el dashboard, permitiendo que la información dinámica reflejada en el panel web quede correctamente registrada.

Este proyecto proporciona una herramienta eficiente para la administración y supervisión de un servidor, optimizando y facilitando la detección de fallos y la toma de decisiones en cuanto al rendimiento del sistema.

Justificación del proyecto

Actualmente, la mayoría de las compañías, empresas y departamentos necesitan servidores de diversa índole para proveer de servicios a usuarios tanto internos como externos. La gestión eficiente de servidores y la supervisión en tiempo real de los recursos y servicios de red es un reto crucial para los administradores de sistemas (Turnbull, 2017). Por ello, tiene especial relevancia disponer de una herramienta sencilla, intuitiva e integral que permita realizar un monitoreo en tiempo real de los servicios, la red y los recursos de un servidor (García, 2020).

En este contexto en el que la necesidad de garantizar una disponibilidad y rendimiento óptimos de un servidor resulta fundamental, nace la idea de este proyecto: crear un panel de administración web interactivo y funcional que permita a los administradores de sistemas visualizar el estado y rendimiento de los servidores en tiempo real, optimizando la toma de decisiones y facilitando la resolución de problemas técnicos.

El propósito es ofrecer una plataforma que no solo proporcione una interfaz amigable para los administradores, sino que también almacene registros detallados sobre el estado y el rendimiento del servidor, lo que facilitará tanto el análisis histórico como la optimización de recursos a lo largo del tiempo.

Actualmente, existen en el mercado distintas soluciones de código abierto que permiten realizar esta monitorización que persigue este proyecto. Entre ellas, Netdata ofrece soluciones para la supervisión de servidores y sus servicios específicos (Netdata, 2023), y Webmin, proporciona un servicio muy similar (Webmin, 2023), al igual que en el caso de Zabbix, que se posiciona como una solución a nivel empresarial (Zabbix, 2023).

Las principales similitudes, diferenciaciones y añadidos que se pretenden desarrollar en este proyecto quedan recogidas en la siguiente tabla:

Característica	Proyecto	Zabbix	Netdata	Webmin
Código abierto	✓	✓	✓	✓
Monitoreo en tiempo real	✓	✓	✓	✗
Registro histórico	✓	✓	✗	✗
Facilidad de uso e instalación	✓	✗	✓	✓
Diseño sencillo y visual	✓	✗	✓	✗

Este proyecto pretende enfocarse en la facilidad de uso, la personalización y la integración con tecnologías ampliamente conocidas, poniendo énfasis en la simplicidad, la personalización y la integración de bases de datos para el análisis a largo plazo. Muchas de las soluciones actuales no integran directamente una base de datos para el almacenamiento histórico, lo que limita la capacidad de análisis de rendimiento a largo plazo, funcionalidad que dentro de este proyecto en particular si se contempla incorporar.

El público objetivo principal de este proyecto son administradores de sistemas y profesionales de IT que necesitan supervisar servidores web y de red de forma eficiente. Estos usuarios suelen tener un conocimiento técnico avanzado, pero buscan una herramienta sencilla, accesible y eficiente que les permita gestionar múltiples aspectos del servidor sin complicaciones adicionales.

Este panel de administración también puede ser útil para pequeñas y medianas empresas que no cuentan con los recursos o la infraestructura para implementar soluciones de monitoreo más complejas o costosas. Asimismo, puede ser una herramienta útil para equipos de soporte técnico que necesiten supervisar el

estado de los servidores a lo largo del tiempo, registrar métricas de rendimiento y tomar decisiones informadas sobre la optimización de recursos.

A través de un panel web intuitivo, los administradores podrán visualizar y gestionar el estado de los recursos y servicios de un servidor de manera rápida y eficiente, mejorando tanto la eficiencia operativa como la capacidad de respuesta ante incidentes.

Introducción

En un entorno tecnológico cada vez más dependiente de servidores y sistemas en red, la supervisión constante del estado y rendimiento de estos equipos adquiere especial relevancia. Las empresas, ya sean grandes o pequeñas, requieren herramientas que permitan a sus administradores de sistemas tener un control total sobre los servicios, recursos y estado de sus servidores, con el fin de detectar fallos, prevenir caídas y garantizar un rendimiento óptimo.

La mayoría de las soluciones del mercado actual suelen ser complejas y costosas, o requieren de conocimientos técnicos avanzados para su correcta configuración y uso. Por este motivo, surge la idea de desarrollar una herramienta que sea amena, personalizable, fácil de instalar y entender.

Este proyecto propone el desarrollo de un panel de administración web que permita visualizar en tiempo real distintos aspectos del sistema de un servidor Linux: uso de CPU, memoria, estado de servicios como Apache, tráfico HTTP, entre otros. Esta solución no solo presenta la información en una interfaz clara y visual, sino que también registra y almacena los datos en una base de datos, permitiendo su análisis a largo plazo.

Objetivos

En base al desarrollo e ideación de este proyecto, estos serían los objetivos estructurados siguiendo el esquema RFTP (requisitos, funciones, tareas y pruebas:

R – Requisitos

Estos son los objetivos expresados en lenguaje coloquial, lo que se espera que haga el sistema:

1. Visualizar en una página web el estado en tiempo real de un servidor Linux.
2. Monitorear automáticamente recursos del sistema como CPU, RAM, disco, red y servicios.
3. Almacenar los registros obtenidos en una base de datos.
4. Permitir la consulta histórica de los datos de monitoreo.
5. Notificar caídas o errores en servicios o recursos críticos.

F – Funciones

Desglose técnico de las funcionalidades para cumplir los requisitos:

1. Monitorización del hardware
    - Script bash obtiene uso de CPU, frecuencia, carga media, RAM, swap, disco, uso de I/O, red (latencia, paquetes, caídas, ancho de banda).
2. Almacenamiento de datos
    - Base de datos MySQL creada y gestionada con DBeaver. Tablas para CPU, RAM, red, servicios, etc.

3. Interfaz web
    - HTML + CSS para visualización, JavaScript para interacción dinámica.
4. Backend con PHP
    - Inserta los datos obtenidos por los scripts en la base de datos y genera las visualizaciones.
5. Conexión DB ↔ Dashboard
    - PHP conecta la base de datos con la interfaz web para mostrar información en tiempo real y almacenada.

T – Tareas

Tareas detalladas asociadas a cada función:

1. Escritura de scripts de monitoreo (bash)
    - Obtener métricas del sistema operativo y servicios.
    - Ejecutarlos de forma periódica (cron).
2. Diseño e implementación de la base de datos
    - Crear tablas bien normalizadas para cada categoría de datos (CPU, RAM, red...).
    - Validar inserción correcta de datos.
3. Desarrollo del frontend
    - Construir el dashboard con HTML, CSS y JS.
    - Mostrar los valores en gráficos o tablas.
4. Programación backend en PHP
    - Incluir scripts dentro de páginas web.
    - Automatizar el registro y consulta de métricas.

5. Integración total del sistema
    - Verificar comunicación fluida entre scripts, base de datos y visualización web.

P – Pruebas

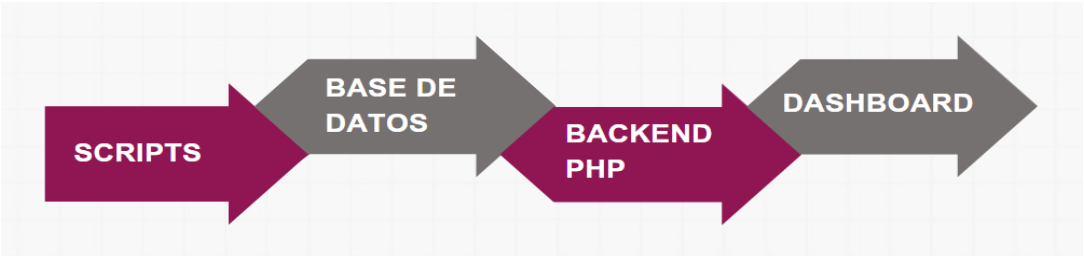
Cómo se demostrará que cada tarea cumple con su función:

1. Prueba de scripts de monitoreo
    - Ejecutar manualmente para validar salida esperada.
    - Probar automatización con cron para confirmación periódica.
2. Prueba de base de datos
    - Verificar inserción, actualización y consulta de datos.
    - Revisar estructura con DBeaver.
3. Prueba del dashboard web
    - Visualización en tiempo real.
    - Navegación fluida y responsive.
4. Prueba de conexión DB ↔ PHP ↔ Web
    - Validar que los datos recolectados se muestran correctamente.
    - Introducir errores deliberados para probar mensajes y control.

Descripción

Uso del proyecto

El sistema consiste en una solución integral para la monitorización de servidores Linux. Mediante scripts Bash se recopilan métricas en tiempo real sobre el estado del sistema (CPU, RAM, red, disco, servicios como Apache…). Estos datos se almacenan en una base de datos MySQL, desde donde se recuperan y visualizan a través de un panel web interactivo.



Un caso de uso típico de este proyecto podría consistir en uno similar al que queda recogido en la siguiente tabla.

Caso de uso	Ver estado de los recursos de un servidor
Actor	Administrador de sistemas
Precondición	El sistema está operativo. Scripts en ejecución.
Flujo principal	1. El administrador accede al panel. 2. Visualiza las gráficas y tablas de uso.
Postcondición	El administrador obtiene una visión clara del estado actual del servidor.

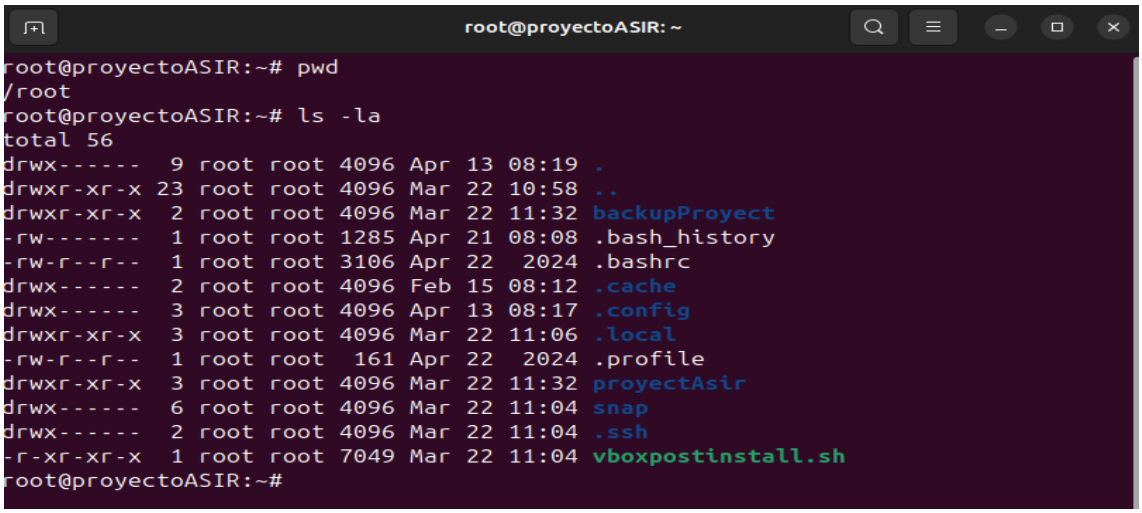
Plan de desarrollo del proyecto

Mediante el siguiente diagrama se pretende resumir las distintas fases del proceso de desarrollo que se han seguido durante la implementación del proyecto.

Fase 1: Scripts Bash

Dentro del sistema Ubuntu se ha creado el directorio proyectASIR, donde a su vez se ha creado el directorio scripts, lugar en que se almacenan cada uno de los distintos comandos y scripts que se emplean durante la fase de desarrollo y cuyos resultados se reflejarán en el dashboard web.

Se puede ver representado desde esta captura realizada en el propio sistema.



```
root@proyectoASIR: ~
root@proyectoASIR:~# pwd
/root
root@proyectoASIR:~# ls -la
total 56
drwx------  9 root root 4096 Apr 13 08:19 .
drwxr-xr-x 23 root root 4096 Mar 22 10:58 ..
drwxr-xr-x  2 root root 4096 Mar 22 11:32 backupProyect
-rw-------  1 root root 1285 Apr 21 08:08 .bash_history
-rw-r--r--  1 root root 3106 Apr 22  2024 .bashrc
drwx------  2 root root 4096 Feb 15 08:12 .cache
drwx------  3 root root 4096 Apr 13 08:17 .config
drwxr-xr-x  3 root root 4096 Mar 22 11:06 .local
-rw-r--r--  1 root root  161 Apr 22  2024 .profile
drwxr-xr-x  3 root root 4096 Mar 22 11:32 proyectAsir
drwx------  6 root root 4096 Mar 22 11:04 snap
drwx------  2 root root 4096 Mar 22 11:04 .ssh
-r-xr-xr-x  1 root root 7049 Mar 22 11:04 vboxpostinstall.sh
root@proyectoASIR:~#
```

En la siguiente imagen se pueden visualizar todos los scripts que se han desarrollado y que han quedado alojados dentro de este directorio de scripts.

```
root@proyectoASIR:~/proyectAsir/scripts# ls -la
total 68
drwxr-xr-x 2 root root 4096 Apr 21 08:22 .
drwxr-xr-x 3 root root 4096 Mar 22 11:32 ..
-rwxr-xr-x 1 root root  255 Apr 13 08:20 backup.sh
-rwxrwxrwx 1 root root  405 Apr 21 08:22 checkConnection.sh
-rwxrwxrwx 1 root root  550 Apr 21 08:20 dispUSB.sh
-rwxrwxrwx 1 root root  297 Apr 13 08:14 DNSserv.sh
-rwxrwxrwx 1 root root  269 Apr 13 07:53 fileSist.sh
-rwxrwxrwx 1 root root   14 Apr 13 08:12 IPservDetail.sh
-rwxrwxrwx 1 root root   12 Apr 13 08:08 IPserv.sh
-rwxrwxrwx 1 root root    7 Apr 21 08:15 lastReboot.sh
-rwxrwxrwx 1 root root    7 Apr 21 08:16 listModules.sh
-rwxrwxrwx 1 root root  660 Apr 13 07:49 netInterfaces.sh
-rwxrwxrwx 1 root root   21 Apr 21 08:17 os.sh
-rwxrwxrwx 1 root root    9 Apr 13 07:43 ram.sh
-rwxrwxrwx 1 root root  229 Apr 21 08:19 runProcess.sh
-rwxrwxrwx 1 root root   10 Apr 21 08:14 timeOnline.sh
-rwxrwxrwx 1 root root   90 Apr 13 08:17 useCPU.sh
root@proyectoASIR:~/proyectAsir/scripts#
```

A continuación, se procede a explicar el contenido y objetivo de cada script utilizado.

- useCPU.sh

Se introduce este comando dentro de un script para poder visualizar en tiempo real el uso de CPU.

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    useCPU.sh
#!/bin/bash
# Muestra el uso de la CPU en tiempo real con top
top -b -n 1 | grep "Cpu(s)"
```

- top

  Es una herramienta en tiempo real que muestra el uso del sistema: procesos, CPU, memoria, etc.

- -b

  Significa modo por lotes (batch mode). Esto le dice a top que no abra la interfaz interactiva, sino que imprima la información en la terminal (ideal para scripts).

- -n 1

  Le dice a top que se ejecute solo una vez (una "instantánea"). Sin esto, top seguiría actualizando indefinidamente.

- | grep "Cpu(s)"

  Esto filtra la salida de top para mostrar solo la línea que contiene "Cpu(s)", que es donde aparece el resumen del uso de la CPU.

- timeOnline.sh

Utilizado para saber cuánto tiempo lleva encendido el sistema, sin información extra.

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    timeOnline.sh
uptime -p
```

  - uptime

    Muestra cuánto tiempo ha estado encendido el sistema.

  - -p

    Muestra ese tiempo en un formato más amigable, legible por humanos.

- runProcess.sh

Este script te da una lista de procesos que actualmente están usando activamente la CPU

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    runProcess.sh
#!/bin/bash

# Script para mostrar los procesos en estado "running"

echo "==== PROCESOS EN ESTADO 'RUNNING' ====="
echo ""

# Mostrar procesos cuyo estado es 'R' (running) usando ps
ps -eo pid,ppid,user,stat,cmd | awk '$4 ~ /R/'
```

ps -eo pid,ppid,user,stat,cmd

Este comando muestra todos los procesos activos del sistema, con columnas personalizadas:

  - pid

    ID del proceso.

  - ppid

    ID del proceso padre.

  - user

    Usuario dueño del proceso.

  - stat

    Estado del proceso (aquí es donde vemos si está "running", "sleeping", etc.).

  - cmd

    El comando que inició el proceso.

- ram.sh

Este comando muestra un resumen del uso de la memoria RAM y la memoria swap del sistema.

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    ram.sh
free -h
```

  - Free

    Muestra información sobre la memoria del sistema.

  - -h

    Significa “human readable”, es decir, los valores se mostrarán en MB, GB, etc. en lugar de bytes crudos (más fácil de leer).

- os.sh

Da una forma rápida de saber qué distribución estás usando (Ubuntu, Debian, CentOS, etc.) y que versión exacta es.

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    os.sh
cat /etc/os-release
```

  - cat

    Muestra el contenido de un archivo.

  - /etc/os-release

    Es un archivo del sistema que contiene información del sistema operativo.

- netInterfaces.sh

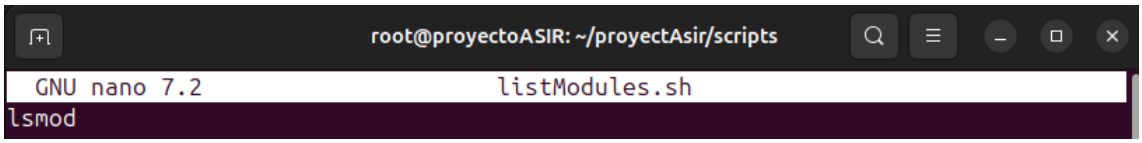
Este script muestra todas las interfaces de red del sistema (como eth0, lo, enp0s3, wlan0, etc.) y su estado: si están activas ("UP") o caídas ("DOWN").

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    netInterfaces.sh
#!/bin/bash

# Mostrar todas las interfaces de red y su estado (activas o caídas)
echo "Estado de las interfaces de red en el sistema:"

# Obtener todas las interfaces de red (filtrando líneas que contienen 'state')
ip -o link show | awk -F': ' '{print $2}' | while IFS= read -r interface
do
    # Obtener el estado de la interfaz (UP, DOWN, etc.)
    state=$(ip link show "$interface" | grep -oP '(?<=state )\w+')

    # Verifica si se obtuvo el estado
    if [[ -z "$state" ]]; then
        echo "$interface: Estado desconocido"
    elif [[ "$state" == "UP" ]]; then
        echo "$interface: Activa"
    else
        echo "$interface: Caída"
    fi
done
```

- listModules.sh

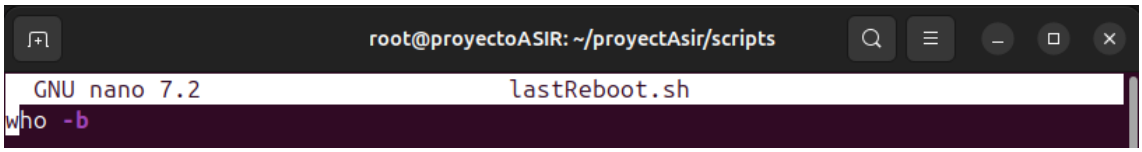
Elabora una lista todos los módulos del kernel actualmente cargados en el sistema Linux**.**

Un módulo es un componente del kernel que puede cargarse o descargarse en tiempo de ejecución, como drivers para hardware, sistemas de archivos, protocolos de red, etc.



- lastReboot.sh

Muestra cuándo fue la última vez que el sistema fue arrancado.

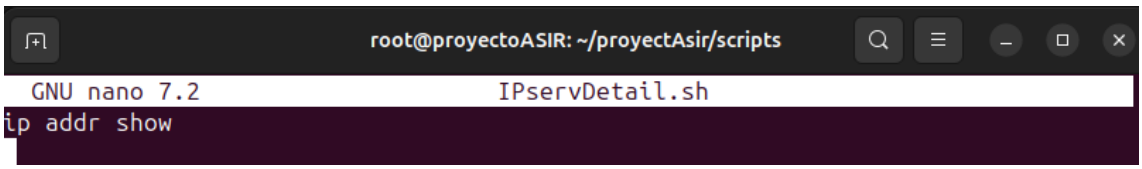


- IPserv.sh

Saber qué IP tiene tu servidor sin ver toda la salida.

- IPservDetail.sh

Este comando muestra toda la información de las interfaces de red en el sistema, incluidas las direcciones IP asignadas, las interfaces de red activas, la configuración de red, etc.



- fileSist.sh

Muestra el porcentaje de uso de los sistemas de archivos del sistema, filtrando solo las particiones principales (que comienzan con /dev).

Presenta la salida en un formato legible y fácil de entender.

- df -h –total
  - df

    Muestra información sobre el uso del sistema de archivos (espacio usado y libre en las particiones).
  - -h

    Muestra el resultado en formato legible por humanos (con unidades como GB, MB, etc.).
  - --total

    Incluye un resumen de todos los sistemas de archivos al final (como un total).
- grep -E '^/dev'
  - grep -E

    Filtra la salida usando expresiones regulares extendidas.
  - '^/dev'

    Solo muestra las líneas que comienzan con /dev, que es típicamente donde se encuentran las particiones del disco en Linux (por ejemplo, /dev/sda1).

- awk '{print $1 ": " $5 " de uso"}'

  Este filtro elimina cualquier información sobre sistemas de archivos de tipo tmpfs o de tipo virtual.

  - Awk

    Filtra y formatea la salida.

  - $1

    Es la primera columna, que corresponde al nombre del sistema de archivos (por ejemplo, /dev/sda1).

  - $5

    Es la quinta columna, que corresponde al porcentaje de uso del sistema de archivos.

  - "de uso"

    Imprime el texto "de uso" junto al porcentaje.

- DNSserv.sh

Para saber el dominio de búsqueda predeterminado en un sistema

```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    DNSserv.sh
#!/bin/bash

# Leer el dominio de /etc/resolv.conf
DOMAIN=$(grep "^search" /etc/resolv.conf | awk '{print $2}')

# Verificar si se encontró el dominio
if [ -z "$DOMAIN" ]; then
    echo "No se pudo encontrar el dominio en /etc/resolv.conf."
else
    echo "El dominio del servidor es: $DOMAIN"
fi
```

  - DOMAIN=$(grep "^search" /etc/resolv.conf | awk '{print $2}')

    - grep "^search" /etc/resolv.conf

      Busca la línea que comienza con search en el archivo /etc/resolv.conf. Este archivo contiene información sobre la configuración de DNS en Linux.

    - Search

      Es una línea que suele contener el dominio de búsqueda, como un sufijo que se agrega a los nombres de dominio no completamente calificados (FQDN).

- awk '{print $2}'

Extrae el segundo campo de la línea encontrada, que es el dominio que sigue a la palabra search.

- if [-z "$DOMAIN"]; then
  - [-z "$DOMAIN"]

Verifica si la variable DOMAIN está vacía (es decir, si no se encontró ningún dominio en el archivo /etc/resolv.conf).

- -z

Comprueba si el valor de la variable es vacío.

- echo "No se pudo encontrar el dominio en /etc/resolv.conf."
  - Si DOMAIN está vacío, imprime el mensaje:
  - "No se pudo encontrar el dominio en /etc/resolv.conf."
  - else
  - Si DOMAIN no está vacío (es decir, se encontró un dominio), entonces:
- echo "El dominio del servidor es: $DOMAIN"
  - Imprime el dominio que se encontró en el archivo /etc/resolv.conf.

- dispUSB.sh

Realiza una serie de comprobaciones relacionadas con los discos conectados, específicamente a dispositivos USB como pendrives, y su estado de montaje en el sistema.

- lsblk -o NAME,TRAN,SIZE,MOUNTPOINT,LABEL,MODEL | grep -E 'usb|^sd'

  - lsblk

    Muestra todos los dispositivos de bloque en el sistema (discos duros, particiones, unidades USB, etc.).

  - -o NAME,TRAN,SIZE,MOUNTPOINT,LABEL,MODEL

  Personaliza la salida para que muestre las siguientes columnas:

    - NAME → Nombre del dispositivo (como sda, sdb1, etc.).
    - TRAN → Tipo de transporte (como usb o sata).
    - SIZE → Tamaño del dispositivo.
    - MOUNTPOINT → El punto de montaje (si está montado).
    - LABEL → Etiqueta del dispositivo, si tiene una.
    - MODEL → Modelo del dispositivo.

  - grep -E 'usb|^sd'

  Filtra la salida para mostrar solo los dispositivos USB (aquellos con "usb" en la columna TRAN) o los dispositivos de tipo SATA/SCSI (comienza con sd).

- mount | grep "^/dev"

  - mount

  Muestra todos los sistemas de archivos montados actualmente en el sistema.

  - grep "^/dev"

  Filtra la salida para mostrar solo los dispositivos que están montados (que generalmente se encuentran bajo /dev, como /dev/sda1, /dev/sdb1, etc.).

- checkConnection.sh

Sirve para verificar si una IP o dominio está accesible desde tu servidor.

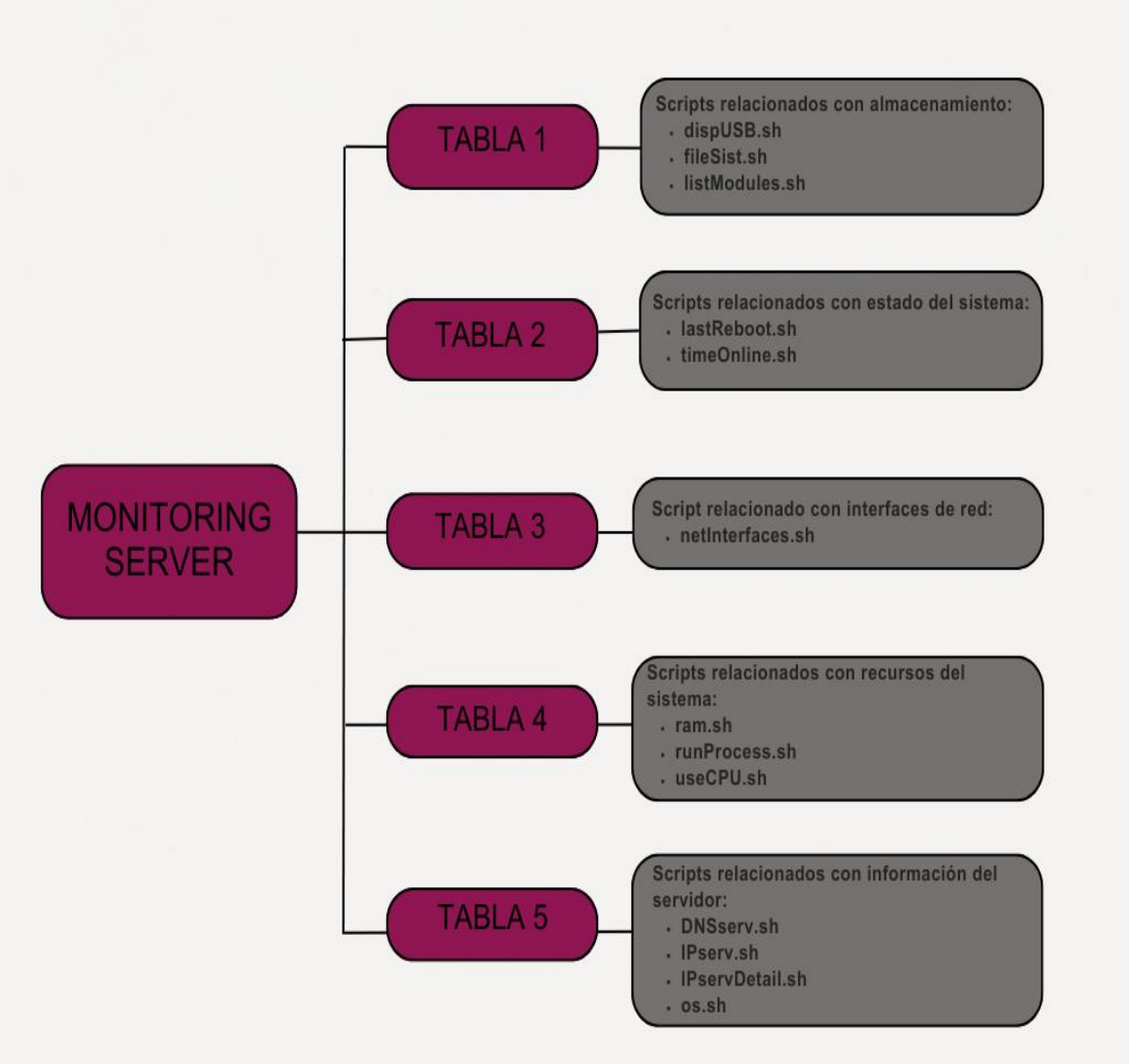
```
root@proyectoASIR: ~/proyectAsir/scripts
  GNU nano 7.2                    checkConnection.sh
#!/bin/bash

# Script para comprobar la conexión a una IP proporcionada

read -p "Introduce la IP o dominio a comprobar: " DESTINO

echo "Comprobando conexión a $DESTINO ..."
echo ""

# Ping con 4 paquetes
ping -c 4 $DESTINO

# Verificar si fue exitoso
if [ $? -eq 0 ]; then
    echo ""
    echo "Conexión exitosa a $DESTINO"
else
    echo ""
    echo "No se pudo establecer conexión con $DESTINO"
fi
```

- read -p "Introduce la IP o dominio a comprobar: " DESTINO
  - Pide al usuario que escriba una IP o dominio (por ejemplo, 8.8.8.8 o google.com).
  - Lo que el usuario escribe se guarda en la variable DESTINO.
- echo "Comprobando conexión a $DESTINO ..."
  - Muestra un mensaje informando que se va a hacer la comprobación.
- ping -c 4 $DESTINO
  - Ejecuta el comando ping para enviar 4 paquetes a la IP o dominio que el usuario ingresó.
  - -c 4: significa que solo se enviarán 4 paquetes ICMP (ping), y luego terminará.
- if [$? -eq 0]; then
  - Aquí se evalúa el código de salida del comando anterior (ping).
  - $? contiene el código de salida del último comando ejecutado.
  - Si el ping tuvo éxito, devuelve 0 (significa que hubo respuesta).
  - Si falló (host no disponible, IP incorrecta, red caída, etc.), devuelve otro número.

Fase 2: Base de datos en MySQL

Se ha empleado Dbeaver para la creación de la base de datos en la cual se almacena la información monitoreada del servidor siguiendo los apuntes proporcionados por la UAX (IAW, 2025 y GBD, 2024). Esta, se ha llamado monitoring_server.

El desarrollo y diseño de las tablas se ha realizado atendiendo al tipo de información que proporciona cada script. Información que proviene de scripts que tienen una funcionalidad similar son almacenados en una misma tabla. Siguiendo esta idea se han desarrollado estas cinco tablas:



Estas serían las sentencias en lenguaje SQL que se han creado en Dbeaver durante la realización de las tablas:

- TABLA 1: almacenamiento

```sql
CREATE TABLE `almacenamiento` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `dispositivos_usb` text DEFAULT NULL,
  `uso_disco` text DEFAULT NULL,
  `modulos_kernel` text DEFAULT NULL,
  `fecha_registro` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

  - CREATE TABLE almacenaje:

    Crea una nueva tabla llamada almacenamiento.

  - id int(11) NOT NULL AUTO_INCREMENT:

    Define la columna id como un entero de 11 dígitos, que no puede ser nulo. Además, AUTO_INCREMENT significa que su valor se incrementará automáticamente cada vez que se inserte una nueva fila.

  - dispositivos_usb text DEFAULT NULL:

    Define una columna de tipo text para almacenar información sobre los dispositivos USB. El valor por defecto es NULL.

  - uso_disco text DEFAULT NULL:

    Define una columna de tipo text para almacenar el uso del disco. El valor por defecto es NULL.

  - modulos_kernel text DEFAULT NULL:

    Define una columna de tipo text para almacenar los módulos del kernel. El valor por defecto es NULL.

  - fecha_registro timestamp NULL DEFAULT current_timestamp():

    Define una columna de tipo timestamp` que almacena la fecha y hora de registro. El valor por defecto es la fecha y hora actual cuando se crea la fila.

  - PRIMARY KEY (id):

    Declara que la columna id es la clave primaria de la tabla, lo que significa que debe ser única para cada fila.

  - ENGINE=InnoDB:

    Especifica que la tabla usará el motor de almacenamiento InnoDB, que soporta transacciones y claves foráneas.

- DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci:

Define el conjunto de caracteres y la intercalación de la tabla como utf8mb4, que es adecuado para almacenar texto multilingüe, incluyendo caracteres emoji y otros caracteres especiales.

- TABLA 2: estado_sistema

```
CREATE TABLE `estado_sistema` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `ultimo_reinicio` datetime DEFAULT NULL,
  `tiempo_actividad` varchar(100) DEFAULT NULL,
  `fecha_registro` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

- id int(11) NOT NULL AUTO_INCREMENT:

Igual que en la tabla anterior, id es un campo entero de 11 dígitos que se incrementa automáticamente y no puede ser nulo.

- ultimo_reinicio datetime DEFAULT NULL:

Define una columna de tipo datetime que almacena la fecha y hora del último reinicio del sistema. Puede ser NULL si no se proporciona un valor.

- tiempo_actividad varchar(100) DEFAULT NULL:

Define una columna de tipo varchar(100) para almacenar el tiempo de actividad del sistema (en formato de texto, como por ejemplo "2 días, 3 horas").

- fecha_registro timestamp NULL DEFAULT current_timestamp()`:

Similar a la tabla anterior, almacena la fecha y hora de cuando se registró la fila.

- PRIMARY KEY (id):

La columna id es la clave primaria de la tabla.

- ENGINE=InnoDB y DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci:

Establecen el motor de almacenamiento InnoDB y el conjunto de caracteres utf8mb4.

- TABLA 3: interfaces_red

```
CREATE TABLE `interfaces_red` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `interfaz_nombre` varchar(100) DEFAULT NULL,
  `estado` varchar(100) DEFAULT NULL,
  `ip_asiganda` varchar(100) DEFAULT NULL,
  `fecha_registro` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

  - id int(11) NOT NULL AUTO_INCREMENT:

Igual que en las tablas anteriores, id es un campo entero único que se incrementa automáticamente.

  - interfaz_nombre varchar(100) DEFAULT NULL:

Define una columna de tipo varchar(100) que almacena el nombre de la interfaz de red (como "eth0", "wlan0", etc.). El valor por defecto es NULL.

  - estado varchar(100) DEFAULT NULL:

Define una columna de tipo varchar(100) para almacenar el estado de la interfaz de red (como "activo", "desconectado", etc.).

  - ip_asiganda varchar(100) DEFAULT NULL:

Define una columna de tipo varchar(100) para almacenar la dirección IP asignada a la interfaz de red.

  - fecha_registro timestamp NULL DEFAULT current_timestamp()`:

Almacena la fecha y hora de cuando se registró la fila, con el valor por defecto como la hora actual.

  - PRIMARY KEY (id):

La columna id es la clave primaria de la tabla.

  - ENGINE=InnoDB y DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci:

Establecen el motor de almacenamiento InnoDB y el conjunto de caracteres utf8mb4.

- TABLA 4: recursos_sistema

```
CREATE TABLE `recursos_sistema` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `cpu_uso` varchar(100) DEFAULT NULL,
  `ram_total` text DEFAULT NULL,
  `procesos_activos` text DEFAULT NULL,
  `fecha_registro` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

  - id int(11) NOT NULL AUTO_INCREMENT:

Similar a las tablas anteriores, id es un campo entero que se autoincrementa.

  - cpu_uso varchar(100) DEFAULT NULL:

Define una columna de tipo varchar(100) que almacena el uso de la CPU en formato de texto (por ejemplo, "75%").

  - ram_total text DEFAULT NULL:

Define una columna de tipo text para almacenar información sobre la memoria RAM total.

  - procesos_activos text DEFAULT NULL:

Define una columna de tipo text que almacena información sobre los procesos activos del sistema.

  - fecha_registro timestamp NULL DEFAULT current_timestamp()`:

Almacena la fecha y hora en que se registró la fila.

  - PRIMARY KEY (id):

La columna id es la clave primaria de la tabla.

  - ENGINE=InnoDB y DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci:

Establecen el motor de almacenamiento InnoDB y el conjunto de caracteres utf8mb4.

- TABLA 5: red_servidor

```
CREATE TABLE `red_servidor` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `dns_dominio` varchar(200) DEFAULT NULL,
  `ip_servidor` varchar(200) DEFAULT NULL,
  `ip_detail_servidor` text DEFAULT NULL,
  `sist_operativo` varchar(200) DEFAULT NULL,
  `version_so` varchar(200) DEFAULT NULL,
  `fecha_registro` timestamp NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

  - id int(11) NOT NULL AUTO_INCREMENT:

Al igual que en las otras tablas, id es un campo entero que se autoincrementa.

  - dns_dominio varchar(200) DEFAULT NULL:

Define una columna de tipo varchar(200) que almacena el nombre de dominio DNS del servidor.

  - ip_servidor varchar(200) DEFAULT NULL:

Define una columna de tipo varchar(200) que almacena la dirección IP del servidor.

  - ip_detail_servidor text DEFAULT NULL:

Define una columna de tipo text para almacenar detalles adicionales sobre la IP del servidor.

  - sist_operativo varchar(200) DEFAULT NULL:

Define una columna de tipo varchar(200) que almacena el nombre del sistema operativo del servidor.

  - version_so varchar(200) DEFAULT NULL:

Define una columna de tipo varchar(200) que almacena la versión del sistema operativo del servidor.

  - fecha_registro timestamp NULL DEFAULT current_timestamp()`:

Almacena la fecha y hora en que se registró la fila.

  - PRIMARY KEY (id):

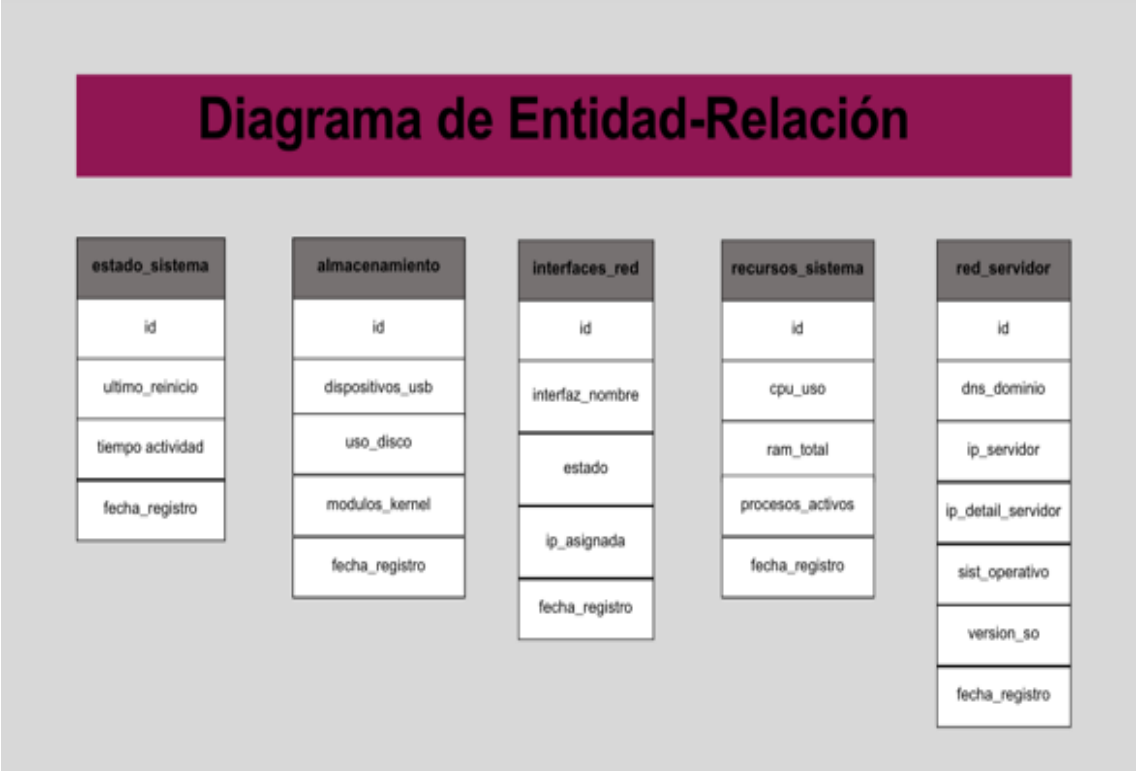
La columna id es la clave primaria de la tabla.

  - ENGINE=InnoDB y DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci:

Establecen el motor de almacenamiento InnoDB y el conjunto de caracteres utf8mb4.

Estas sentencias quedan también recogidas en un documento aparte llamado anexo 1.

Así es el grafo del diagrama entidad relación de la base de datos tras la creación de las tablas:



Fase 3: Panel Web

Mediante Visual Studio Code se ha desarrollado el código que constituye el dashboard que visualizamos en el navegador web para observar la información proporcionada por los scrips.

Este sería el esquema base que sigue la estructura de directorios y archivos del panel web:

- CSS
  - Carpeta que contiene la hoja de estilos de index.php. Permite customizar y personalizar la estética del panel web.
- Img
  - Carpeta cuya función es actuar como banco de imágenes que son utilizadas para la personalización del dashboard.

- Scripts
  - Carpeta en la que se almacenan todos los scripts (creados en la fase 1) que permiten acceder a la información pertinente para la monitorización del servidor según se ha diseñado.

- Actualizardatos.php
  - Archivo que contiene las funciones necesarias para ejecutar los scripts en el sistema.
- Conexion.php
  - Contiene las funciones necesarias para que se dé la conexión con la base de datos y poder realizar el registro de información proveniente de los scripts.
- Index.php
  - Archivo principal del proyecto encargado de mostrar la información y solicitar la actualización de la misma.
- Insertardatos.php
  - Archivo que realiza las inserciones de información en la base de datos cada que vez que el dashboard es abierto en el navegador.
- Index.js
  - Contiene las funciones de JavaScript necesarias para la implementación.

Así queda resumida la estructura de directorios y archivos propios del panel web:

Toda esta estructura queda almacenada dentro de la siguiente ruta en el sistema Ubuntu: /opt/lampp/apache2/htdocs/dashboardProyecto/index.php.

Se ha almacenado dentro del dirctorio htdocs propio de Apache ya que este será el servidor que aloja el panel web para poder ser visualizado en el navegador web.

Todo el código que compone esta estructura de directorios y archivos queda adjuntado en un conjunto de archivos aparte denominado anexo 2.

Fase 4: Conexión a base de datos

Se crea el archivo conexion.php y se realizan las adaptaciones necesarias en el resto de archivos que conforman el dashboard para que la información recabada por los scripts se muestre en el panel web y estos datos sean recogidos y almacenados por la base de datos.

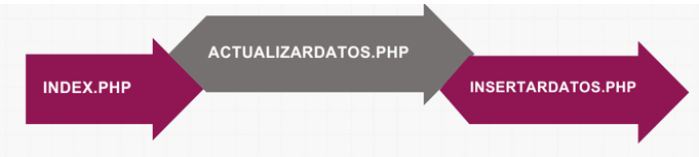
Este archivo conexión.php recoge el siguiente código para permitir esta funcionalidad descrita:

conexion.php ×

C: > xampp2 > htdocs > proyecto > dashboardProyecto > conexion.php

```php
<?php
// Este script se conecta a una base de datos MySQL usando la extensión mysqli

// Definición de las credenciales y parámetros de conexión

// ejecutar la funcion mysqli() -> conectarme a una base de datos con unos permisos determinados
// servidor al que me conecto -> localhost
// usuario (root) pass () database (monitoring_server)


const username = "root";    // Nombre de usuario para acceder a la base de datos
const password = "";        // Contraseña del usuario (en este caso vacía)
const database = "monitoring_server";     // Nombre de la base de datos a la que se desea conectar
const host = "localhost";                 // Servidor de base de datos (localhost al estar en el mismo servidor)

// Se crea una nueva variable de conexión MySQLi usando los parámetros definidos
$connection = new mysqli(host, username, password, database);
// Función que verifica si se ha producido un error durante la conexión
if ($connection->connect_error) {
    echo "error en conexion";   // Muestra mensaje simple si hay error
    die("Conexión errónea: " . $connection->connect_error);  // Detalla el error y detiene la ejecución
}
?>

```

El flujo de datos que sigue la información recabada por los scripts se representa mediante este esquema, donde se observa una clara separación de responsabilidades entre los archivos:



1. El usuario abre index.php o lo recarga.
2. index.php incluye actualizardatos.php.
3. actualizardatos.php se ejecuta:
    - Ejecuta los scripts.
    - Limpia la salida.
    - Usa insertarDato() (de insertardatos.php) para insertar los datos en recursos_sistema.
4. index.php continúa ejecutándose obteniendo la información de los scripts y mostrándola constantemente.

Tecnologías

Servidor Ubuntu

Descripción

Para el desarrollo e implementación del proyecto de monitorización, se ha optado por utilizar un sistema operativo basado en Linux, concretamente Ubuntu Server, debido a su robustez, eficiencia y compatibilidad con herramientas de administración. Según Nemeth et al. (2017), Linux destaca por su estabilidad en entornos de producción, permitiendo tiempos de actividad prolongados sin necesidad de reinicios, lo cual es esencial en servidores que requieren disponibilidad continua.

Además, al tratarse de un sistema operativo de código abierto, ofrece la posibilidad de adaptar y optimizar su funcionamiento sin costes de licencia, lo que representa una ventaja significativa frente a otras soluciones propietarias (Turnbull, 2017).

Desde el punto de vista práctico, Linux ofrece compatibilidad nativa con herramientas de monitorización y automatización como Bash, cron o systemctl, lo que ha facilitado la integración directa de los scripts desarrollados en este proyecto. Esta decisión se alinea también con las recomendaciones académicas para sistemas de supervisión, como expone García (2020), donde se concluye

que Linux es especialmente adecuado para entornos de monitorización por su bajo consumo de recursos y su alto grado de personalización.

Uso en el proyecto

Durante todo el proyecto, se emplea una máquina virtual Ubuntu Server Desktop 24.04.2 en Virtual Box con las siguientes características:

- Disco duro: 95GB
- RAM: 8GB

Bash y Shell Scripting

Descripción

Bash (*Bourne Again SHell*) es uno de los intérpretes de comandos más populares en los sistemas operativos tipo Unix, especialmente en entornos Linux. Este shell no solo permite la interacción directa con el sistema operativo a través de una interfaz de línea de comandos, sino que también admite la ejecución de scripts automatizados, lo que lo convierte en una herramienta idónea para tareas de administración, monitoreo y automatización de procesos (Free Software Foundation, 2022).

Bash es compatible con el shell original *sh*, e incorpora características avanzadas provenientes de otros shells como *ksh* (Korn Shell) y *csh* (C Shell). Está diseñado para cumplir con los estándares IEEE POSIX P1003.2/ISO 9945.2 sobre shell y herramientas, lo que asegura su portabilidad y consistencia en diferentes entornos Unix (GNU Project, 2023).

Entre las principales características que ofrece Bash se incluyen:

- Edición de línea de comandos en tiempo real.
- Historial de comandos sin límite de tamaño.
- Control de trabajos en segundo plano.
- Definición de funciones y alias.
- Soporte para matrices indexadas de tamaño ilimitado.
- Capacidad para realizar operaciones aritméticas en cualquier base entre 2 y 64 (Free Software Foundation, 2022).

El shell scripting, permite automatizar tareas mediante la escritura de comandos en archivos de texto, conocidos como scripts. Estos scripts son interpretados y ejecutados por el shell, lo que facilita la implementación de trabajos repetitivos o complejos de forma eficiente (GNU Project, 2023).

Algunos conceptos fundamentales en shell scripting son:

- Shebang (#!). Línea inicial de un script que indica al sistema qué intérprete utilizar, por ejemplo #!/bin/bash.
- Variables. Permiten almacenar y manipular información dentro del script, como números, cadenas o listas.
- Sentencias condicionales. Permiten ejecutar distintas acciones según ciertas condiciones, como con if, else y elif.
- Bucles. Permiten repetir bloques de código, siendo los más comunes for, while y until.
- Estado de salida. Cada comando devuelve un código que indica si se ejecutó correctamente (0) o si ocurrió un error (códigos diferentes de cero).
- Comentarios. Usados para documentar el código. Se indican con el símbolo #, y son ignorados durante la ejecución (Free Software Foundation, 2022).

Uso en el proyecto

Su uso se da en la fase 1 (Scripts Bash) para crear los scripts que recogen información de estado, recursos y servicios del servidor.

Dbeaver

Descripción

DBeaver (2023) nos explica que es una herramienta de gestión de bases de datos de código abierto, gratuita y multiplataforma, diseñada para facilitar el trabajo de desarrolladores, administradores de bases de datos, analistas de datos y cualquier profesional que precise de sistemas de almacenamiento de información. Su objetivo principal es ofrecer una experiencia de usuario altamente funcional y accesible, centrada en la usabilidad.

DBeaver es compatible con la mayoría de las bases de datos SQL populares, tales como MySQL, MariaDB, PostgreSQL, SQLite, Oracle, SQL Server, entre otras. Esto es posible gracias a su arquitectura basada en controladores JDBC, lo que también le permite integrarse con fuentes de datos externas, incluso si no cuentan con un controlador JDBC específico.

Entre sus principales características destacan:

- Interfaz gráfica intuitiva y moderna, que permite manipular datos de forma similar a una hoja de cálculo.
- Editor SQL avanzado, que incluye autocompletado, resaltado de sintaxis y ejecución de scripts complejos.
- Gestión de esquemas y migración de datos, útil para tareas de administración y mantenimiento de bases de datos.
- Generación de diagramas ERD (Entity-Relationship Diagrams), facilitando la visualización de estructuras de bases de datos.
- Exportación e importación de datos en múltiples formatos.
- Creación de informes analíticos a partir de diferentes fuentes de datos.
- Supervisión de sesiones de conexión a bases de datos activas.
- Compatibilidad multiplataforma, funcionando en sistemas Windows, macOS y Linux.
- Integración con herramientas y extensiones populares como Git y Excel.

- Asistencia impulsada por inteligencia artificial para optimizar tareas y sugerir mejoras.
- Soporte para plugins personalizados, lo cual amplía su funcionalidad de forma modular.

Permite manipular los datos como si trabajara en una hoja de cálculo normal, crear informes analíticos a partir de registros de varios almacenamientos de datos y exportar la información en el formato deseado. Para usuarios avanzados de bases de datos, DBeaver ofrece un potente editor SQL, amplias funciones de administración, herramientas para la migración de datos y esquemas, supervisión de sesiones de conexión a bases de datos y mucho más.

Centrado en la facilidad de uso, DBeaver ofrece:

- Interfaz de usuario intuitiva y moderna
- Funciones de seguridad de nivel empresarial
- Compatibilidad con una amplia gama de bases de datos
- Perfecta integración con fuentes de datos en la nube
- Compatibilidad con extensiones para Excel, Git, etc
- Asistencia impulsada por IA
- Multitud de herramientas para la gestión y el análisis de bases de datos
- Compatibilidad multiplataforma para Windows, macOS y Linux.

Uso de la herramienta en el proyecto

Se emplea durante la fase 2 (Base de datos) para el diseño y gestión de la base de datos MySQL. El propósito de su uso es crear y modificar tablas (ej. cpu_usage, apache_logs), poder validar la estructura de la base de datos y realizar consultas de prueba.

Apache Server

Descripción

El Proyecto Servidor HTTP Apache es un esfuerzo colaborativo de desarrollo de software cuyo objetivo es crear una implementación de código fuente de un servidor HTTP (Web) robusto, de calidad comercial, lleno de características y disponible gratuitamente. El proyecto está gestionado conjuntamente por un grupo de voluntarios de todo el mundo que utilizan Internet y la Web para

comunicarse, planificar y desarrollar el servidor y la documentación relacionada (The Apache Software Foundation, 2023).

Su principal función es gestionar y entregar páginas web a los usuarios a través del protocolo HTTP (Hypertext Transfer Protocol).

Apache actúa como intermediario entre el navegador del usuario y los archivos del sitio web que están almacenados en un servidor. Cuando un usuario solicita una página web (por ejemplo, al escribir una URL en el navegador), Apache recibe esa solicitud, localiza el recurso correspondiente en el servidor y lo envía al navegador para que el usuario pueda visualizarlo.

Algunas de sus características más destacadas son:

- Multiplataforma. Funciona en sistemas operativos como Linux, Windows y macOS, entre otros.
- Modularidad. Permite añadir o quitar funcionalidades mediante módulos, como los módulos de seguridad, compresión, reescritura de URLs, entre otros.
- Compatibilidad con múltiples lenguajes. Soporta lenguajes de servidor como PHP, Python, Perl y más, lo que lo hace ideal para aplicaciones dinámicas.
- Configuración flexible. A través de archivos como .htaccess se puede personalizar el comportamiento del servidor por directorios.
- Seguridad y rendimiento. Es capaz de manejar múltiples conexiones simultáneamente de forma eficiente, y permite implementar diversas medidas de seguridad, como autenticación, control de acceso y cifrado SSL/TLS (The Apache Software Foundation, 2023).

Apache es ampliamente utilizado en entornos de desarrollo y producción, y forma parte de muchos stacks de desarrollo como el popular XAMP (Multiplataforma, Apache, MySQL, PHP/Python/Perl).

Según Apache Friends. (2023), XAMPP es una distribución de software libre que agrupa un conjunto de tecnologías necesarias para implementar un servidor web funcional de forma sencilla y multiplataforma. El acrónimo XAMPP hace referencia a los siguientes componentes:

- X – Cross-platform: compatible con varios sistemas operativos (Windows, Linux, macOS).
- A – Apache: servidor HTTP que gestiona las solicitudes web.
- M – MySQL o MariaDB: sistema de gestión de bases de datos relacional.
- P – PHP: lenguaje de programación del lado del servidor.
- P – Perl (opcional en algunos paquetes).

A diferencia del stack LAMP, que se configura manualmente en sistemas Linux, XAMPP ofrece un entorno preconfigurado y portátil, lo cual agiliza el proceso de instalación y prueba de aplicaciones web sin necesidad de administrar cada componente por separado (Meloni, 2006; Smith, 2023).

Uso en el proyecto

El momento de uso se da en las fases 3 y 4 con las funciones de: servir los archivos HTML/CSS/JS al cliente, ejecutar scripts PHP y gestionar la comunicación con la base de datos.

PHP

Descripción

PHP (Hypertext Preprocessor) se trata de un lenguaje de código abierto interpretado, de alto nivel, embebido en páginas HTML y ejecutado en el servidor, muy popular y especialmente adecuado para el desarrollo web (PHP Group, 2023). El código PHP es ejecutado en el servidor, el cliente solo recibe el resultado de su ejecución, sin ninguna posibilidad de determinar que código ha producido el resultado obtenido (Meloni, 2006).

PHP puede ser usado en cualquiera de los principales sistemas operativos del mercado, Linux, Unix, Microsoft Windows, MAC OS X...

Soporta la mayoría de servidores web de hoy en día, incluyendo Apache, IIS, Personal Web Server, Netscape e iPlanet... Además, tiene módulos disponibles para la mayoría de los servidores, para aquellos otros que soporten el estándar CGI, PHP puede usarse como procesador CGI (Nixon, 2018).

Tiene la libertad de elegir el SO y el servidor de su gusto. También tiene la posibilidad de usar programación procedimental o programación orientada a objetos. Aunque no todas las características estándar de la programación orientada a objetos están implementadas en la actual versión de PHP, muchas bibliotecas y aplicaciones grandes (incluyendo la biblioteca PEAR) están escritas íntegramente usando programación orientada a objetos (Meloni, 2006).

Una de las principales razones de la popularidad de PHP como lenguaje de creación de scripts para Web es su amplio soporte a diferentes bases de datos. Este soporte facilita que los desarrolladores creen sitios sustentados en bases de datos y que se hagan nuevos prototipos de aplicaciones web de manera rápida y eficiente, sin demasiada complejidad.

Quizás la característica más potente y destacable de PHP es su soporte para una gran cantidad de bases de datos. Escribir un interfaz vía web para una base de datos es una tarea simple con PHP (Nixon, 2018).

PHP no se encuentra limitado a resultados en HTML. Entre las habilidades de PHP se incluyen: creación de imágenes, archivos PDF y películas Flash (usando libswf y Ming) sobre la marcha. También puede presentar otros resultados, como XHTM y archivos XML. PHP puede autogenerar estos archivos y almacenarlos en el sistema de archivos en vez de presentarlos en la pantalla (Nixon, 2018).

Existen tres campos en los que se usan scripts escritos en PHP (Nixon, 2018):

- Scripts del lado del servidor. Este es el campo más tradicional y el principal foco de trabajo. Se necesitan tres cosas para que esto funcione: el intérprete PHP (CGI o módulo), un servidor web y un navegador. Es necesario correr el servidor web con PHP instalado. El resultado del programa PHP se puede obtener a través del navegador, conectándose con el servidor web.
- Scripts en la línea de comandos. Puede crear un script PHP y correrlo sin ningún servidor web o navegador. Solamente necesita el intérprete PHP para usarlo de esta manera. Este tipo de uso es ideal para scripts ejecutados regularmente desde cron (en +nix o Linux) o el planificador de tareas (en Windows). Estos scripts también pueden ser usados para tareas simples de procesamiento de texto.

- Escribir aplicaciones de interfaz gráfica. Probablemente PHP no sea el lenguaje más apropiado para escribir aplicaciones gráficas, pero si se conoce bien PHP, y se quisiera usar algunas características avanzadas en programas clientes, se puede usar PGP-GTK para escribir dichos programas. También es posible escribir aplicaciones independientes de una plataforma. PHP-GTK es una extensión de PHP, no disponible en la distribución principal.

Además, puede ser incrustado en HTML, encerrando el código de PHP entre las etiquetas especiales de comienzo y final <?php y ?> que permiten entrar y salir del «modo PHP» (PHP Group, 2023).

Uso de la herramienta en el proyecto

Esta herramienta será utilizada en dos fases del proyecto, saber:

En la fase 3 (Panel Web), para generar dinámicamente el contenido HTML del dashboard.

En la Fase 4 (Conexión a la base de datos), como backend para:

- Recibir datos de los scripts Bash,
- Insertar/consultar datos en MySQL,
- Enviar información a la interfaz web, y
- Conectar el frontend (HTML/JS) con la base de datos.

DHTML

El uso de HTML, hojas de estilo y JavaScript se acostumbra a denominarse HTML dinámico o DHTML (Zakas, 2021)

HTML5

Descripción

HTML (HyperText Markup Language) es el lenguaje de marcado estándar utilizado para crear y estructurar páginas web. Desde su creación en 1989, HTML ha evolucionado constantemente a través de diversas versiones (HTML 2.0, HTML 3.2, HTML 4.0, HTML 4.01), incorporando mejoras que ampliaron su funcionalidad y refinaron su sintaxis.

Con la llegada de HTML5, se introdujo una versión moderna del lenguaje, adaptada a las necesidades actuales de la web. Esta nueva especificación fue diseñada con un enfoque práctico, buscando resolver problemas reales en el desarrollo web y facilitando la creación de aplicaciones interactivas y multimedia.

Entre sus principales aportes se encuentran:

- Incorporación nativa de elementos multimedia, como video (<video>) y audio (<audio>), eliminando la necesidad de utilizar plugins externos como Adobe Flash.
- Mayor rendimiento y capacidad de respuesta, optimizado especialmente para dispositivos móviles y navegadores modernos.
- Nuevas APIs integradas, como geolocalización, almacenamiento local (localStorage y sessionStorage), y funcionalidades de arrastrar y soltar (drag & drop).
- Soporte para gráficos y animaciones mediante elementos como <canvas> y la integración con SVG.
- Mejoras semánticas, con nuevas etiquetas (<article>, <section>, <header>, <footer>, <nav>, etc.) que permiten estructurar el contenido de forma más clara y accesible.
- Facilitación del desarrollo de aplicaciones web avanzadas, especialmente aquellas orientadas a plataformas móviles o entornos de bajo consumo energético.

Los tres objetivos fundamentales de HTML5 son:

1. Especificar los comportamientos reales de los navegadores para asegurar la interoperabilidad.
2. Definir un tratamiento estándar de errores, permitiendo a los navegadores interpretar correctamente el código incluso cuando contiene fallos.
3. Evolucionar el lenguaje HTML para adaptarlo a las exigencias actuales del desarrollo web moderno.

Gracias a estas mejoras, HTML5 se ha consolidado como la base fundamental para la creación de experiencias web ricas, dinámicas y accesibles desde cualquier dispositivo o navegador.

Uso de la herramienta en el proyecto

Durante la fase 3 (Panel Web), proporcionando la estructura básica del dashboard y definiendo la estructura y contenido estático de la interfaz web.

CSS

Descripción

Según explica Meyer (2020). CSS (Cascading Style Sheets) es un lenguaje de hojas de estilo en cascada que se utiliza para definir la presentación visual de los documentos escritos en lenguajes de marcado como HTML, XML o XHTML. Fue desarrollado por primera vez en 1994 y más adelante adoptado por el consorcio W3C como parte de los estándares oficiales del desarrollo web.

El principal objetivo de CSS es separar el contenido y la estructura de un documento web de su diseño visual, lo que permite una mayor flexibilidad, facilidad de mantenimiento y reutilización del código. Esta separación mejora la accesibilidad y usabilidad, ya que facilita la adaptación del contenido a distintos dispositivos, resoluciones de pantalla o preferencias del usuario.

CSS utiliza una sintaxis declarativa y sencilla, basada en reglas formadas por selectores (que apuntan a los elementos HTML que se desean estilizar) y declaraciones (que definen las propiedades visuales, como colores, fuentes, tamaños, márgenes, etc.). Este sistema de reglas se organiza mediante el modelo en cascada, el cual permite combinar estilos desde múltiples fuentes, priorizándolos según criterios específicos (por ejemplo, el orden de carga, la especificidad del selector o el uso de la palabra clave !important).

Además de ser un pilar fundamental en el diseño de sitios web modernos, CSS ha trascendido el entorno web, utilizándose también en interfaces de dispositivos embebidos, clientes de RSS, aplicaciones de mensajería, y como influencia directa en la sintaxis y lógica de frameworks de JavaScript. Esta amplia adopción refleja su versatilidad y la capacidad de adaptarse a entornos diversos.

En resumen, CSS ha evolucionado desde un lenguaje básico de estilos hasta convertirse en una herramienta poderosa y flexible para controlar el aspecto visual de interfaces gráficas en todo tipo de plataformas, garantizando una experiencia coherente, personalizable y eficiente para usuarios y desarrolladores.

Uso de la herramienta en el proyecto

Esencial en la fase 3 (Panel Web) para el diseño visual del dashboard (colores, disposición), y la estabilización de la interfaz para mejorar la usabilidad y estética.

JavaScript

Descripción

Según Zakas (2021) JavaScript es un lenguaje de programación interpretado, ampliamente utilizado en el desarrollo web para dotar de dinamismo e interactividad a las páginas. Fue creado por Netscape en la década de 1990 y hoy en día se basa en el estándar ECMAScript. A diferencia de los lenguajes compilados, JavaScript se ejecuta directamente en el navegador del usuario, lo que permite respuestas rápidas a eventos sin necesidad de recargar la página ni enviar solicitudes al servidor.

JavaScript es esencialmente un lenguaje orientado a eventos, dinámico y basado en prototipos, con un sistema de tipado débil. Estas características lo hacen altamente flexible y accesible para los desarrolladores. Su sintaxis está inspirada en C, y guarda similitudes con Java, aunque sus fundamentos conceptuales son distintos.

Una de sus principales funciones es mejorar la experiencia del usuario mediante la incorporación de elementos interactivos, como animaciones, validaciones de formularios, aparición de mensajes emergentes, o acciones que se activan al pulsar botones. Todo esto es posible gracias a la capacidad de JavaScript para manipular el DOM (Document Object Model) del documento HTML en tiempo real.

JavaScript puede integrarse en una página web de tres maneras principales (Flanagan,2020):

- Insertando código directamente dentro del contenido HTML.
- Definiendo scripts dentro de la etiqueta <head>.
- Enlazando archivos externos con extensión .js, práctica recomendada por facilitar la organización y el mantenimiento del código, al igual que sucede con los archivos CSS.

A pesar de su gran utilidad, uno de los retos de trabajar con JavaScript es que su ejecución puede variar ligeramente entre navegadores, lo que obliga a los desarrolladores a realizar pruebas de compatibilidad para asegurar una experiencia consistente para todos los usuarios.

Además de su tradicional uso en el lado del cliente, hoy en día JavaScript también se utiliza en el lado del servidor gracias a entornos como Node.js, lo que ha ampliado sus posibilidades a campos como el desarrollo de microservicios, aplicaciones en tiempo real y sistemas de alta carga computacional.

Uso de la herramienta en el proyecto

En la fase 3 (Panel Web), para la interactividad del dashboard y asegurar que el panel sea adaptable a diferentes dispositivos

Bootstrap

Descripción

Según W3Schools (2023), Bootstrap es un framework front-end gratuito para un desarrollo web más rápido y sencillo. Incluye plantillas de diseño basadas en HTML y CSS para tipografía, formularios, botones, tablas, navegación, modales, carruseles de imágenes y muchos otros, así como plugins JavaScript opcionales.

En resumidas cuentas, nos permite crear fácilmente diseños adaptables.

Las ventajas del uso de Bootstrap son:

- Fácil de usar. Cualquiera con conocimientos básicos de HTML y CSS puede empezar a utilizar Bootstrap.
- Características responsive. El CSS responsive de Bootstrap se ajusta a teléfonos, tabletas y ordenadores de sobremesa.
- Enfoque mobile-first. En Bootstrap 3, los estilos mobile-first forman parte del núcleo del framework.
- Compatibilidad con navegadores. Bootstrap es compatible con todos los navegadores modernos (Chrome, Firefox, Internet Explorer, Edge, Safari y Opera).

Uso de la herramienta en el proyecto

Durante la fase 3 (Panel Web), con el objetivo de poder simplificar el desarrollo frontend con componentes predefinidos.

Tabla resumen de las tecnologías

TECNOLOGÍA	FASE	USO
Servidor Ubuntu	Preparación del entorno	Servidor sobre el que se realiza el monitoreo
Bash	Fase 1: Scripts	Recolectar métricas del servidor
DBeaver	Fase 2: Base de datos	Almacenar datos estructurados y permitir consultas históricas
Apache	Fase 3: Panel Web Fase 4: Conexión a base de datos	Alojar dashboard web
DHTML, Bootstrap	Fase 3: Panel Web	Estructura y estilo del panel web.
PHP	Fase 3: Panel Web Fase 4: Conexión a base de datos	Gestionar comunicación entre base de datos, dashboard y scripts

Metodología

Según González (2017), una metodología de desarrollo de software es un marco de trabajo que permite estructurar, planificar y controlar el proceso de desarrollo de sistemas de información. Su papel es fundamental en cualquier proyecto tecnológico, ya que establece directrices claras sobre los plazos, el presupuesto, las actividades necesarias y los tiempos de revisión. La correcta elección de la metodología impacta directamente en la eficiencia, calidad y viabilidad del producto final (Leiva Mundaca & Villalobos Abarca, 2015).

En este proyecto se ha optado por una metodología tradicional con enfoque en cascada, debido a la naturaleza modular, secuencial y controlada del sistema que se ha desarrollado. El modelo en cascada, como indican Molina Montero et al. (2018), establece desde el inicio los requisitos, fases y entregables, lo que resulta apropiado en contextos donde el alcance está bien definido y no se esperan cambios frecuentes.

Este enfoque ha permitido dividir el trabajo en tareas bien delimitadas:

1. Diseño de la propuesta del proyecto
2. Fases de la implementación del proyecto comentadas anteriormente:
    - Fase 1: Desarrollo de scripts Bash
    - Fase 2: Creación de la base de datos con DBeaver
    - Fase 3: Desarrollo del panel web con tecnologías frontend y backend
    - Fase 4: Conexión con la base de datos.
3. Documentación del proyecto.

Cada fase era dependiente de la anterior y fueron desarrolladas y probadas por separado antes de pasar a la siguiente. Esta secuencialidad facilita la planificación inicial y el seguimiento posterior de las actividades.

Como se ha indicado en otras secciones, estas son las pruebas secuenciales que se han ido realizando durante la implementación del proyecto:

1. Prueba de scripts de monitoreo
    - Ejecutar manualmente para validar la salida esperada.
    - Probar automatización con cron para confirmación periódica.
2. Prueba de base de datos
    - Verificar inserción, actualización y consulta de datos.
    - Revisar estructura con DBeaver.
3. Prueba del dashboard web
    - Visualización en tiempo real.
    - Navegación fluida y responsive.
4. Prueba de conexión DB ↔ PHP ↔ Web
    - Validar que los datos recolectados se muestran correctamente.
    - Introducir errores deliberados para probar mensajes y control.

Una vez terminado todo el desarrollo se realizan pruebas de caso general de uso, en las que se simula ser un administrador de sistemas de este modo:

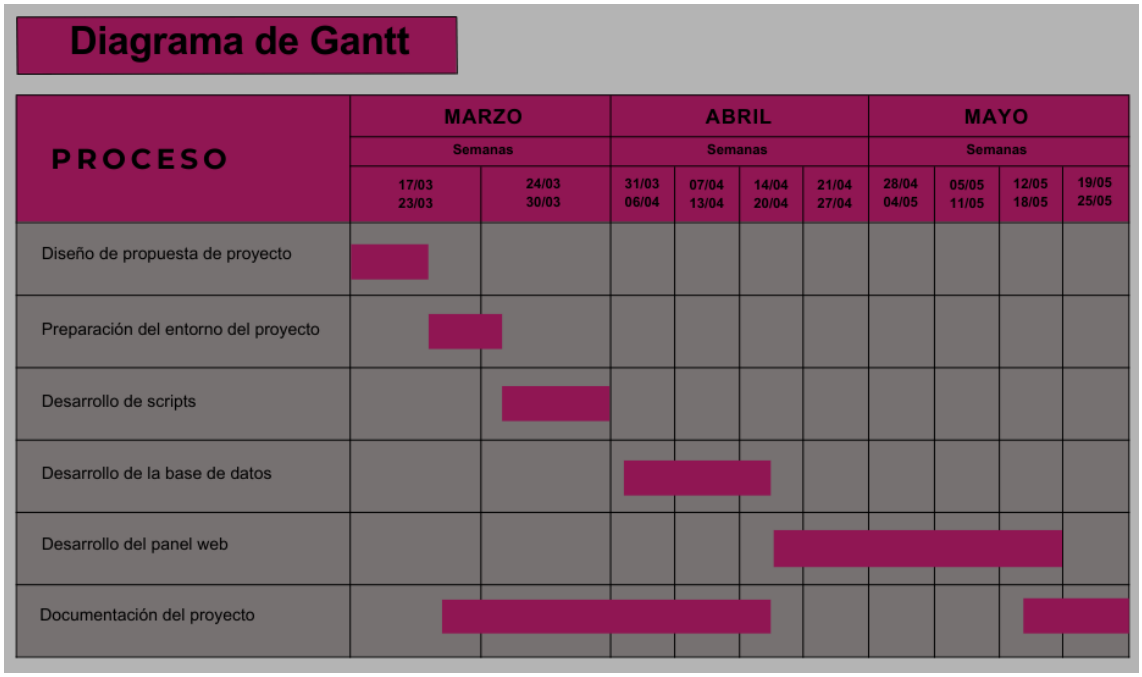
1. Se lanza XAMPP mediante el comando: $ sudo /opt/lampp/manager-linux-x64.run.
    - Al desplegarse panel de control de XAMPP se inician los programas de mysql y apache servidor para que pueda funcionar el dashboard web y la base de datos.
2. Desde la terminal del sistema se abre la aplicación Dbeaver la cual gestiona la base de datos mediante el comando: dbeaver-ce.
3. Para comprobar la visualización del dashboard web se utiliza el navegador web mozilla Firefox y en su barra de búsqueda se escribe: http://localhost/proyecto/dashboardProyecto/index.php.
4. Desde otra pestaña del navegador web se accede a http://localhost/phpmyadmin para comprobar si cada vez que se actualiza la página del dashboard se realiza una nueva inserción de datos en la base de datos monitoring_server.

Según lo propuesto por Leiva Mundaca y Villalobos Abarca (2015), las metodologías tradicionales también permiten una gestión clara de los recursos y el tiempo, lo cual es especialmente valioso cuando se trabaja de forma individual o bajo supervisión académica.

Con motivo de poder organizar y visualizar el proceso de desarrollo del proyecto, se han eleborado dos diagramas de Gantt:

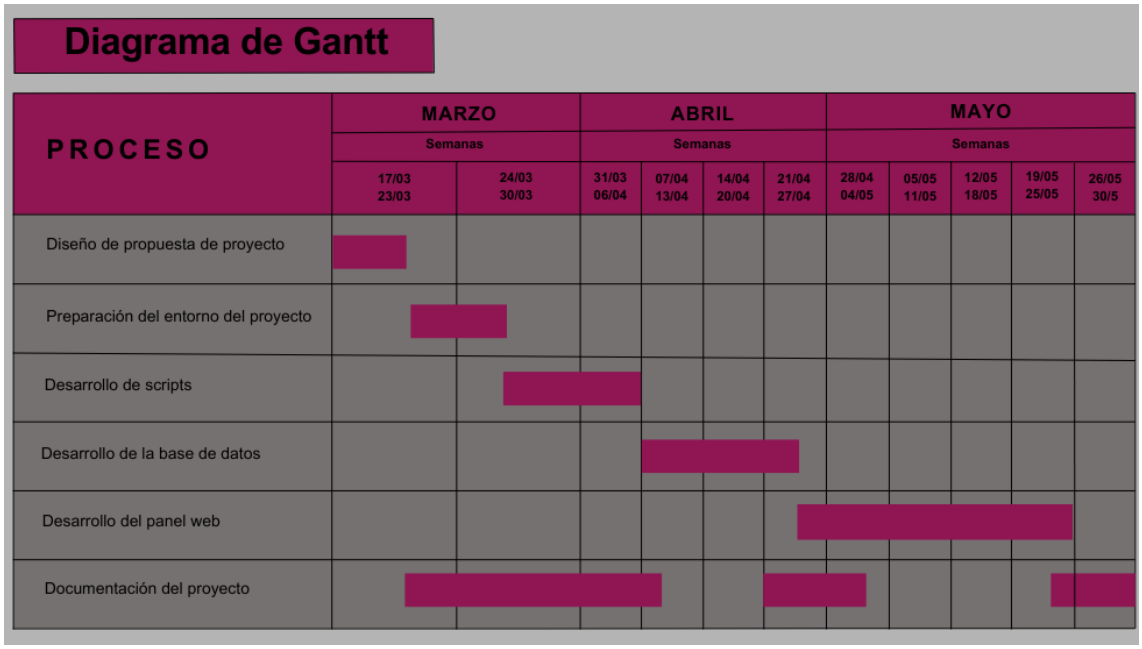
- Planificación inicial:

Este primero, representa una estimación inicial de las tareas y tiempos que iban a conllevar cada una.



- Planificación final:

Este segundo, representa el coste real del desarrollo de cada tarea.



En la siguiente tabla, se puede observar la estimación de las horas que se han dedicado a cada tarea y al proyecto en general:

Fase	Actividades	Horas
Diseño de propuesta de proyecto	Definición de objetivos, alcance, planificación inicial	1.86h
Preparación del entorno del proyecto	Instalación de herramientas, configuración de entorno	2.48h
Desarrollo de scripts	Programación en Bash, pruebas de monitorización, automatización con cron	8.06h
Desarrollo de la base de datos	Modelo entidad-relación, creación de tablas, pruebas de inserción en MySQL.	5.58h
Desarrollo del dashboard web	Frontend (HTML, CSS, JS), Backend (PHP), conexión con base de dato	18.6h
Documentación del proyecto	Redacción del informe, formato, anexos, diagramas y bibliografía	25.42h
Total estimado		62h

Trabajos futuros

Como propuestas de ampliación y mejoras proyectadas quedan recogidos en los siguientes puntos a continuación:

- Finalización del desarrollo del sistema de prueba de conexión y registro de destinos consultados

A través del script desarrollado llamado checkConnection.sh se pretende poder realizar pruebas de conexión a otras direcciones IP o nombres de dominio desde el propio dashboardweb introduciendo aquella dirección que queremos verificar.

Además, se propone almacenar en la base de datos todas las IPs o nombres de dominio utilizados en las pruebas de conectividad. Este registro permitirá llevar un historial de consultas realizadas y servirá como base para análisis posteriores.

Para ello se creará una nueva tabla en la base de datos ya creada en la que se inserten todo este tipo de datos.

- Fortalecer los mecanismos de autenticación y control de acceso al sistema

Para asegurar el acceso a la plataforma de monitorización, se contempla la inclusión de un sistema de autenticación que gestione un listado de usuarios autorizados. Según la OWASP, es esencial implementar controles de autenticación robustos para prevenir accesos no autorizados, incluyendo el almacenamiento seguro de contraseñas mediante algoritmos de hash como bcrypt o Argon2.

- Realizar peticiones AJAX al backend (PHP) para actualizar datos sin recargar la página.

La implementación de peticiones AJAX permitirá una mayor fluidez y experiencia de usuario al actualizar dinámicamente los datos del servidor sin necesidad de recargar la página completa. Esto es especialmente útil para dashboards en tiempo real. Según Mozilla Developer Network (MDN), AJAX permite enviar y recibir datos de forma asíncrona usando tecnologías como JavaScript, XMLHttpRequest y JSON.

- Incorporación de Bootstrap para mejorar la interfaz web

Aunque el dashboard actual cumple su función técnica de visualización de métricas, su diseño ha sido construido de forma manual con HTML y CSS sin el uso de frameworks. Como mejora futura, se plantea integrar el framework Bootstrap con el objetivo de:

  - Mejorar la estética visual y la coherencia del diseño, aplicando estilos modernos y profesionalizados.
  - Optimizar la experiencia de usuario (UX) mediante componentes responsivos y predefinidos.
  - Facilitar la adaptación del panel a distintos dispositivos (ordenadores, tablets, móviles) sin necesidad de reescribir el CSS.

Bootstrap proporciona una amplia colección de componentes reutilizables (botones, tarjetas, menús, formularios) y un sistema de rejillas que permite construir interfaces responsive de manera eficiente (W3Schools, 2023).

Su incorporación no solo permitirá reducir el esfuerzo de desarrollo visual en futuras versiones, sino que también mejorará la accesibilidad y navegabilidad del panel para usuarios finales.

- Migración de XAMPP a instalación directa de servicios

Actualmente, el sistema utiliza XAMPP como entorno integrado de servidor web y base de datos, lo cual es práctico en entornos de desarrollo (Apache Friends, 2023). No obstante, como mejora futura se contempla la instalación directa de Apache y MySQL en el sistema operativo del servidor (por ejemplo, mediante apt en Ubuntu). Esta migración permitiría un mayor control sobre la configuración, una mejor integración con herramientas del sistema, mayor seguridad y un entorno más alineado con prácticas estándar en producción.

- Scripts adicionales para monitorear más datos sobre el servidor como por ejemplo pudiera ser que servicios hay activos en el servidor.

Conclusiones

Una vez finalizado todo el proceso de desarrollo y pruebas se puede afirmar que el dashboard web constituido cumple adecuadamente con su propósito inicial al ser capaz de presentar visualmente de manera fácil y rápida información relativa a la monitorización de sistemas de red, diversos recursos y servicios de un servidor Ubuntu. Además, también se ha conseguido con éxito que estos datos recabados queden registrados y almacenados en una base de datos específicamente diseñada y adaptada para ello.

Esta plataforma permite a los administradores de sistemas visualizar el estado y rendimiento de los servidores en tiempo real, optimizando la toma de decisiones y facilitando la resolución de problemas técnicos. La visualización clara y ordenada de métricas clave del sistema, unida al almacenamiento histórico de los registros, convierte esta herramienta en una solución útil y práctica para entornos que requieren supervisión constante y eficaz.

Durante el desarrollo, uno de los principales desafíos técnicos fue la integración de los scripts Bash con el entorno web mediante PHP. Lograr que los datos obtenidos por los scripts se procesaran correctamente y se visualizaran dinámicamente en el archivo index.php implicó una comprensión profunda del flujo de datos entre el sistema operativo, el backend y el frontend. Este reto supuso un punto de inflexión en el aprendizaje y consolidación de conocimientos técnicos fundamentales para el desarrollo web orientado a sistemas.

En su estado actual, el sistema cuenta con una arquitectura sólida y funcional que lo hace competitivo frente a otras herramientas de monitorización más conocidas, especialmente por su sencillez de uso, modularidad y capacidad de adaptación. Aun así, se reconoce que existen mejoras por implementar, como el sistema de autenticación de usuarios, la actualización dinámica de datos sin recarga mediante AJAX, o una interfaz más refinada con el uso de frameworks como Bootstrap.

En definitiva, este proyecto representa una base robusta sobre la que es posible seguir construyendo, incorporando nuevas funcionalidades y mejorando la experiencia de usuario. El proceso ha resultado altamente enriquecedor desde el punto de vista formativo y técnico, permitiendo integrar múltiples tecnologías de forma coherente y aplicarlas a una necesidad real en el ámbito de la administración de sistemas.

Bibliografía

[1] Turnbull, J. (2017). *Practical Monitoring: Effective Strategies for the Real World*. O'Reilly Media.

[2] García, M. (2020). *Diseño e implementación de un sistema de monitoreo para servidores Linux*. Universidad Politécnica Salesiana. Recuperado de

https://dspace.ups.edu.ec/bitstream/123456789/13474/1/UPS-GT001824.pdf

[3] Netdata. (2023). *Netdata: Real-time performance monitoring*. Recuperado de https://www.netdata.cloud/

[4] Zabbix. (2023). *Zabbix: Enterprise-class monitoring solution*. Recuperado de https://www.zabbix.com/

[5] Webmin. (2023). *Webmin: Web-based system administration*. Recuperado de https://webmin.com/

[6] UAX. (2024). *Gestión de bases de datos*. Universidad Alfonso X el Sabio.

[7] UAX. (2024). *Implantación de aplicaciones web*. Universidad Alfonso X el Sabio.

[8] Bibliografía correspondiente a anexo 2

[8] Smith, R. (2023). *Web Development with PHP and MySQL: A Practical Guide*. Apress.

[9] GNU Project. (2023). *Bash Reference Manual*. Recuperado de

https://www.gnu.org/software/bash/

[10] Free Software Foundation. (2022). *Bash Reference Manual* (versión 5.1). GNU Project. https://www.gnu.org/software/bash/manual/bash.html

[11] DBeaver. (2023). *DBeaver: Universal Database Tool*. Recuperado de

https://dbeaver.io/about/

[12] The Apache Software Foundation. (2023). *About Apache HTTP Server*. Recuperado de

https://httpd.apache.org/ABOUT_APACHE.html

[13] Apache Friends. (2023). *What is XAMPP?*. Recuperado de

https://www.apachefriends.org/faq_stack.html

[14] Ullman, L. (2004). *PHP para el mundo real*. Anaya Multimedia. Recuperado en

https://books.google.es/books?hl=es&lr=&id=y11L7pQfdRsC&oi=fnd&pg=PA1&dq=php&ots=5AfYkzNWVT&sig=Ep26sbGGIENqR-GFtt-MGIU02VY&redir_esc=y#v=onepage&q=php&f=false

[15] PHP Group. (2023). *PHP: Hypertext Preprocessor*. Recuperado de

https://www.php.net/manual/es/intro-whatis.php

[16] Meloni, J. C. (2006). *PHP, MySQL y Apache para principiantes*. Pearson Educación. https://books.google.es/books?hl=es&lr=&id=EJ1t1GsMHbIC

[17] Nixon, R. (2018). *Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5* (5.ª ed.). O'Reilly Media.

https://books.google.es/books?hl=es&lr=&id=lWR5DwAAQBAJ

[18] Zakas, N. C. (2021). *Principios de JavaScript: Escriba un código claro, eficiente y fácil de mantener* (1ª ed.). Anaya Multimedia.

https://books.google.es/books?hl=es&lr=&id=_cy4EAAAQBAJ

[19] W3Schools. (2023). *HTML5 Tutorial*. Recuperado de https://online.wlv.ac.uk/what-is-html5/

[20] Meyer, E. (2020). *CSS: The Definitive Guide*. O'Reilly Media.

[21] Flanagan, D. (2020). *JavaScript: The Definitive Guide*. O'Reilly Media.

[22] W3Schools. (2023). Bootstrap Tutotial. Recuperado de https://www.w3schools.com/bootstrap/bootstrap_get_started.asp

[23] Pérez, J., & Sánchez, A. (2019). *Development of Information Systems*. *Proceedings of the 16th International Information Systems Forum (IISF)*, *3*, 14-19.

[24] González, M. (2017). *Metodologías ágiles para el desarrollo de software*. ProQuest. Recuperado de

https://www.proquest.com/openview/937c0ffcf92bf2c2761d76165f34090e/1

[25] Molina Montero, H., González, J., & Santamaría, R. (2018). *Desarrollo de sistemas de información con metodologías tradicionales*. Redalyc. Recuperado de

https://www.redalyc.org/pdf/373/37326902005.pdf

[26] Leiva Mundaca, J., & Villalobos Abarca, R. (2015). *Metodologías híbridas: un enfoque de desarrollo emergente*. Revista Tecnología e Innovación. Recuperado de https://www.ecorfan.org/bolivia/researchjournals/Tecnologia_e_innovacion/vol2num5/Tecnologia_e_Innovacion_Vol2_Num5_6.pdf

[27] WikiBrief. (2023). *Diagrama de Gantt*. Recuperado de https://esp.wikibrief.org/wiki/Gantt_chart

[28] OWASP. (2023). *Authentication Cheat Sheet*. OWASP Foundation. Recuperado de https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[29] Mozilla Developer Network (2023). *Peticiones de red con JavaScript (AJAX)*. Recuperado de https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting/Network_requests

[30] PHP Group. (2023). *HTTP Authentication with PHP*. Recuperado de https://www.php.net/manual/es/features.http-auth.php

Bibliografía anexo 2

[1] GNU Project. (2023). *Bash Reference Manual*. Recuperado de https://www.gnu.org/software/bash/

[2] OpenAI. (2025). *Imagen generada con ChatGPT para el logo del proyecto*. Herramienta de generación visual de ChatGPT

[3] UAX. (2024). *Apuntes de IAW - Implantación de Aplicaciones Web*. Universidad Alfonso X el Sabio.

[4] UAX. (2024). *Apuntes de Lenguaje de Marcas y Sistemas de Gestión de Información*. Universidad Alfonso X el Sabio.

[5] Gregorie, Y. (s.f.). *HTML, CSS y JavaScript para principiantes*. Domestika. Recuperado de https://www.domestika.org/es/courses/4905-html-css-y-javascript-para-principiantes/course

[6] PHP Group. (2023). *PHP Manual - mysqli::prepare*. Recuperado de https://www.php.net/manual/en/mysqli.prepare.php

[7] PHP Group. (2023). *PHP Manual - shell_exec*. Recuperado de https://www.php.net/manual/en/function.shell-exec.php

[8] PHP Group. (2023). *PHP Manual*. Recuperado de

https://www.php.net/manual/en/

[9] MySQL Developers. (2023). *MySQL Documentation*. Recuperado de

https://dev.mysql.com/doc/

[10] W3Schools. (2023). *PHP Tutorial*. Recuperado de

https://www.w3schools.com/php/

[11] Medium. (2023). *Server Monitoring with PHP and Bash Scripts*. Recuperado de

https://medium.com/winkhosting/server-monitoring-with-php-and-bash-scripts-c078616a6e2f

[12] GeeksforGeeks. (2024). *PHP shell_exec vs exec Function*. Recuperado de https://www.geeksforgeeks.org/php-shell_exec-vs-exec-function/

[13] GitHub. (2023). *PHP Server Monitor*. Recuperado de

https://github.com/phpservermon/phpservermon

[14] Reddit. (2013). *Any way to safely execute shell scripts from PHP?*. Recuperado de

https://www.reddit.com/r/PHPhelp/comments/1b0r8ez/any_way_to_safely_execute_shell_scripts_from_php/

[15] McLibre. (2023). *PHP: Expresiones regulares*. Recuperado de

https://www.mclibre.org/consultar/php/lecciones/php-expresiones-regulares.html

[16] Devart. (2023). *MySQL ORDER BY Tutorial*. Recuperado de

https://blog.devart.com/mysql-order-by-tutorial-html.html

[17] PHP Group. (2023). *PHP Control Structures: foreach*. Recuperado de

https://www.php.net/manual/en/control-structures.foreach.php

[18] W3Schools. (2023). *PHP foreach Loop*. Recuperado de

https://www.w3schools.com/php/php_looping_foreach.asp